

## DRY RUN

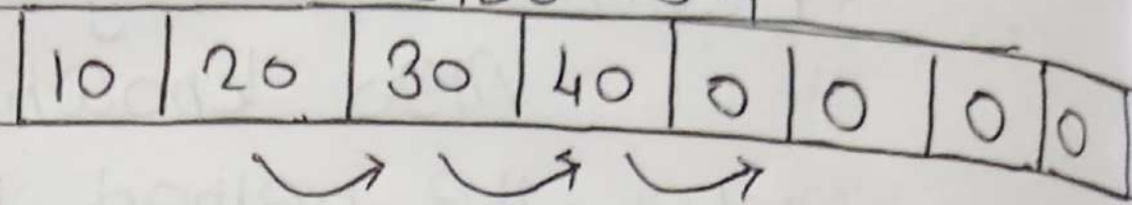
### ArrayBasic. Java

int[] a = {10, 20, 30, 40, 0, 0, 0, 0}

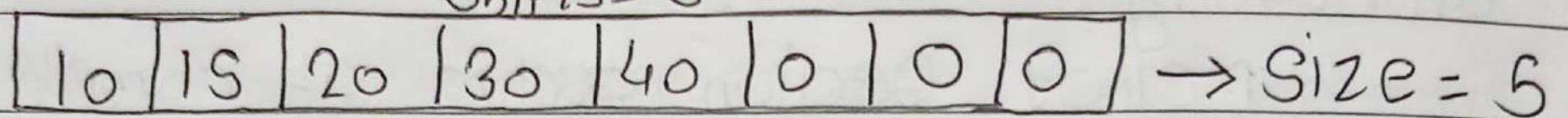
size = 8

① Insert at :- a[1] = 15

Index 1



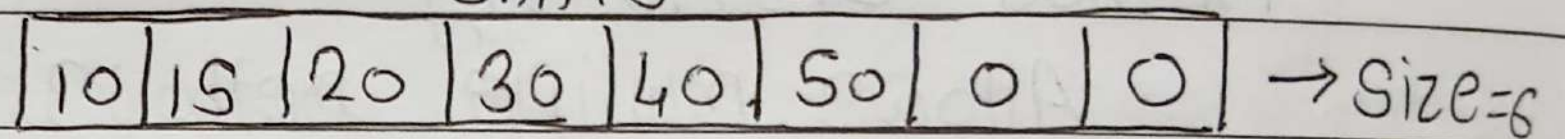
Shifts = 3



② Insert at :-

ending index

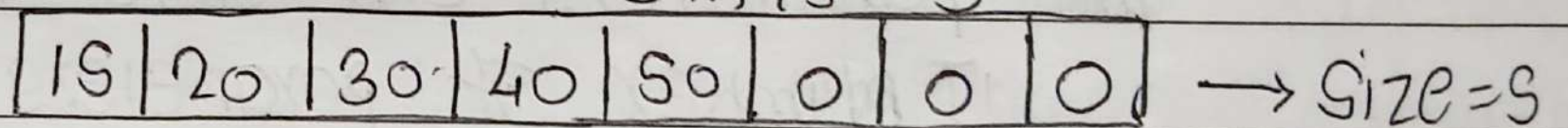
Shifts = 0



③ Delete at

index 0

Shifts = 5



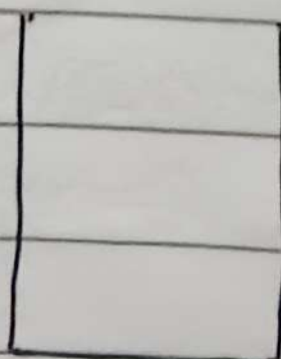
for a[8] = 99;

ArrayOutOfBoundException

because, we only have a[7].

### ArrayStack. Java

Initially the stack is empty



① Push (10)

|    |
|----|
|    |
| 10 |

← top

② Push (20)

|    |
|----|
| 20 |
| 10 |

← top

③ Push (30)

|    |
|----|
| 30 |
| 20 |
| 10 |

← top

④ Peek

|    |
|----|
| 30 |
| 20 |
| 10 |

← top

⑤ Pop 30

|    |
|----|
| 20 |
| 10 |

← top

⑥ Push (40)

|    |
|----|
| 40 |
| 20 |
| 10 |

← top

⑦ Push (50)

|    |
|----|
| 50 |
| 40 |
| 20 |
| 10 |

← top

⑧ Push (60) = overflow  
top = 3  
data.length = 4  
top == data.length - 1

while (!s.is empty)

Pop - 50 ← here follows LIFO

Pop - 40

Pop - 20

Pop - 10

Pop - underflow

## StackDemo. Java

| Character | operation | Stack |
|-----------|-----------|-------|
| {         | Push      | {     |
| [         | Push      | {[    |
| (         | Push      | {[(   |
| )         | Pop ( →   | {[    |
| ]         | Pop [     | {     |
| }         | Pop {     | Empty |

Stack = Empty

## LinearQueue. Java

□ Front = 0, rear = -1

Enqueue

10, 20, 30, 40, 50

10 | 0 | 0 | 0 | 0

front = 0, rear = 0 ← Insert from rear end.

10 | 20 | 0 | 0 | 0

front = 0, rear = 1

10 | 20 | 30 | 0 | 0

front = 0, rear = 2

10 | 20 | 30 | 40 | 0

front = 0, rear = 3

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 20 | 30 | 40 | 50 |
|----|----|----|----|----|

front = 0

rear = 4

② Dequeue 10, 20, 30

|   |   |   |    |    |
|---|---|---|----|----|
| 0 | 0 | 0 | 40 | 50 |
|---|---|---|----|----|

front = 3

rear = 4

③ Enqueue = 60

rear = max - 1

Queue overflow.

5. JavaQueuesDemo.java

Initially

Stack = empty

Push ("Page 1", "Page 2", "Page 3")

|        |       |
|--------|-------|
| Page 3 | ← top |
| Page 2 |       |
| Page 1 |       |

Pop -

|        |        |
|--------|--------|
|        | Page 3 |
| Page 2 |        |
| Page 1 |        |

Second POP C)

Page 1

Page 2

Final POP C)

Page 1

ArrayDeque as Queue

Initially,

Queue = empty

offer ("Job1")

[Job1]

offer ("Job2")

[Job1, Job2]

offer ("Job3")

[Job1, Job2, Job3]

First Poll C)

Poll C)  $\rightarrow$  Job1

[Job2, Job3]

Second Poll (c)

[Job 3]

Output:-

Printer Queue :- Job 1 then Job 2

3. Dequeue - Double ended Queue

Front



[1] [2] [3]

Rear



Initially, dq = empty

① dq.offerLast(2);

[2] →

② (1)

[1, 2]

③ (3)

[1, 2, 3]

Now

Front → [1] [2] [3] ← Rear

Pop J9. PollLast()

Removes - 3

[1, 2]

PopFirst()

Removes - 1

[2]

Output → Deque: [1, 2, 3] PollLast = 3 PollFirst =